

Mini-HPC Cluster Implementation: MPI and Spark-Based Distributed Computing on a Virtual Environment

Name	ID
Jessica Amgad Anis	231001218
Abdalrahman Mohamed Abdo	231001347
Merna Ashraf Daneal	221000689

Table of Contents

Abstract	1
1 Introduction	1
2 Objectives	2
3 Steps 1–2: Virtual Machines and Networking Setup	2
4 Network Troubleshooting Note	3
5 Step 3: Passwordless SSH Setup	4
6 Step 4: OpenMPI and Python Environment Setup	5
7 Step 5: Hostfile Configuration	5
8 Dataset Description	6
8.1 Digits Dataset	6
8.2 Gene Expression Dataset (leukemia_expression.csv)	6
9 Step 6: Distributed Machine Learning Job	6
10 Step 7: Distributed Bioinformatics Job	7
11 Step 8: Docker Installation and Docker Swarm Setup	8
12 Step 9: Spark Cluster Deployment Using Docker Swarm	8
13 Step 10: Distributed Spark Bioinformatics Machine Learning Job	9
14 Results Analysis	10
14.1 Summary of Reported Results	10

14.2	Discussion of Perfect Scores	10
15	Limitations	11
15.1	VM-Based Cluster	11
15.2	No Execution Time Measurement	11
15.3	Missing Screenshots	11
15.4	Username Inconsistency in SSH Commands	11
16	MPI Troubleshooting Note	12
17	Conclusion	12
18	Final Project Files	13
19	References	13

. Abstract

This project involved building a Mini-HPC cluster using three Ubuntu Server 20.04 virtual machines on Oracle VirtualBox — one master node and two worker nodes. The nodes were connected over a private Host-only network (192.168.56.0/24) with passwordless SSH configured for automatic job dispatch from the master.

Two computing frameworks were used. The first was OpenMPI with mpi4py, used to run two distributed Python machine learning jobs: one on the scikit-learn digits dataset and one on a gene expression dataset (leukemia_expression.csv). The second framework was Docker Swarm combined with Apache Spark (spark:3.5.0), used to deploy a distributed Spark cluster and run a PySpark bioinformatics job.

All jobs ran successfully across the cluster and returned scores, confirming that the distributed infrastructure functioned as intended.

1.Introduction

High Performance Computing (HPC) is used to divide large computational tasks across multiple machines so that the work can be completed faster and more efficiently. In this project, a small HPC cluster was built using three Ubuntu Server virtual machines. The cluster included one master node and two worker nodes. The master node was responsible for coordinating the distributed jobs, while worker1 and worker2 were used as compute nodes. The machines were connected through a private Host-only network, and passwordless SSH was configured to allow the master node to launch jobs on the workers automatically.

The main objective of this project was to build a working Mini-HPC cluster and test it using two distributed Python jobs with OpenMPI and mpi4py. The first job used a digit classification dataset, while the second job used a gene expression dataset to demonstrate a simple bioinformatics machine learning workflow.

2. Project Objectives

The main objectives of this project were:

- Create three Ubuntu Server 20.04 VMs (master, worker1, worker2) using Oracle VirtualBox with consistent hardware settings.
- Configure a Host-only private network (192.168.56.0/24) and verify connectivity between all nodes using ping.
- Set up passwordless SSH from the master node to both worker nodes using RSA key pairs.
- Install OpenMPI, Python 3, and the required libraries (mpi4py, scikit-learn, pandas, numpy) on all three nodes.
- Create a hostfile defining 6 total MPI slots (2 per node) and run a distributed machine learning job using the digits dataset.
- Run a second distributed MPI job using a gene expression dataset (leukemia_expression.csv) to demonstrate a bioinformatics workflow.

- Install Docker on all nodes, initialise a Docker Swarm cluster, and add worker nodes using join tokens.
- Deploy Apache Spark (master + 2 worker replicas) as a Docker Swarm stack using a spark-stack.yml file.
- Submit a distributed PySpark machine learning job inside the Spark cluster and verify successful execution.

3.Steps 1–2: Virtual Machines and Networking Setup

Three virtual machines were created using Oracle VirtualBox to build the basic Mini-HPC cluster environment. The virtual machines were named according to their roles in the cluster:

master
worker1
worker2

Each virtual machine was installed with Ubuntu Server 20.04 LTS. The same basic hardware configuration was used for all nodes to keep the cluster setup consistent:

RAM:	4096	MB
CPU:	2	processors
Disk	size:	20 GB
Disk	type:	VDI
Storage: Dynamically allocated		

The master node was prepared as the main coordinator node, while worker1 and worker2 were prepared as compute worker nodes. During the Ubuntu installation, OpenSSH Server was enabled because it will be needed later for remote login and MPI communication between the nodes.

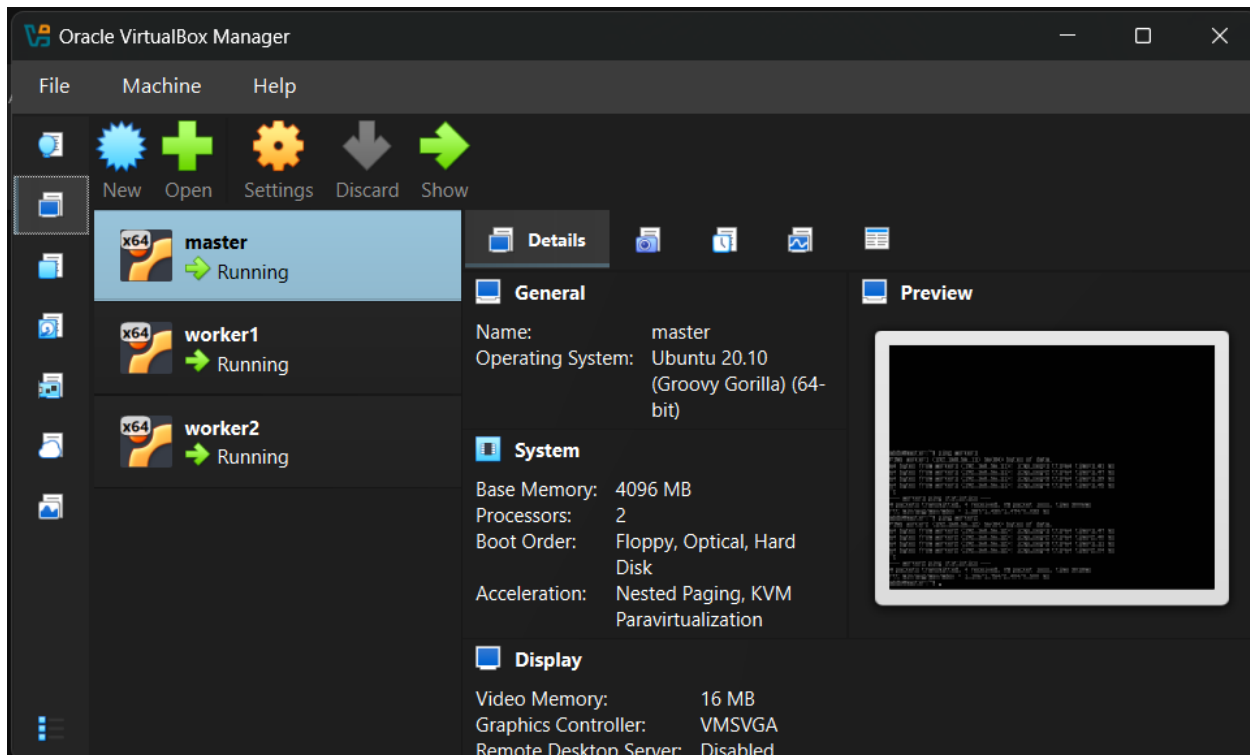


Figure 1: VirtualBox showing the three created VMs: master, worker1, and worker2.

After the operating system installation was completed, the network configuration was prepared. A Host-only Adapter was used to allow the virtual machines to communicate with each other through a private internal network. The Host-only network used the following range: **192.168.56.0/24**

The network adapters were configured so that each VM had access to the private cluster network. The Host-only Adapter was connected to the interface used for communication between the nodes.

The assigned cluster IP addresses were:

master	→	192.168.56.10
worker1	→	192.168.56.11
worker2	→	192.168.56.12

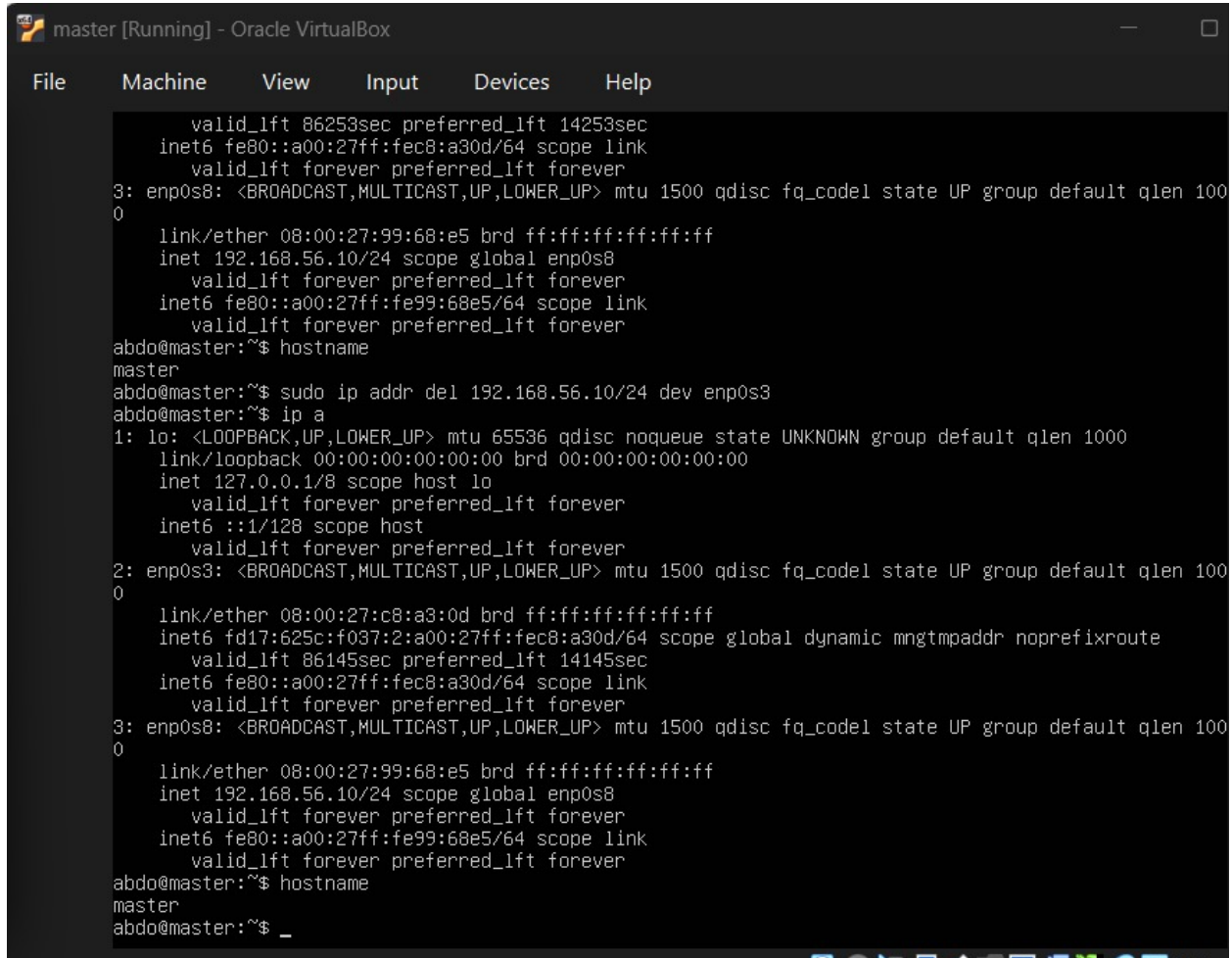
The network interfaces were checked using the `ip a` command. The Host-only interface used for node-to-node communication was `enp0s8`. After configuration and troubleshooting, the assigned IP addresses appeared on the nodes as follows:

master → 192.168.56.10/24

worker1 → 192.168.56.11/24

worker2 → 192.168.56.12/24

This confirmed that each node had a unique address on the private cluster network.



```
valid_lft 86253sec preferred_lft 14253sec
inet6 fe80::a00:27ff:fec8:a30d/64 scope link
    valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:99:68:e5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe99:68e5/64 scope link
        valid_lft forever preferred_lft forever
abdo@master:~$ hostname
master
abdo@master:~$ sudo ip addr del 192.168.56.10/24 dev enp0s3
abdo@master:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c8:a3:0d brd ff:ff:ff:ff:ff:ff
    inet6 fd17:625c:f037:2:a00:27ff:fec8:a30d/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86145sec preferred_lft 14145sec
    inet6 fe80::a00:27ff:fec8:a30d/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:99:68:e5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe99:68e5/64 scope link
        valid_lft forever preferred_lft forever
abdo@master:~$ hostname
master
abdo@master:~$ _
```

Figure 2: Output of ip a on the master node showing 192.168.56.10/24.

The /etc/hosts file was then updated on all three virtual machines. This step was done so that each node could identify the other nodes by hostname instead of using only IP addresses.

The following entries were added to /etc/hosts on master, worker1, and worker2:

192.168.56.10	master
192.168.56.11	worker1
192.168.56.12	worker2

This allowed the machines to communicate using hostnames, such as ping worker1 and ping worker2, instead of writing the full IP address each time. Network connectivity was tested from the master node to both worker nodes. the connection was tested using the IP addresses:

```
ping 192.168.56.11
ping 192.168.56.12
```

Both tests were successful and showed: 0% packet loss

After updating /etc/hosts, hostname-based ping tests were also performed:

```
ping worker1
ping worker2
```

These tests confirmed that name resolution was working correctly and that the master node could reach both worker nodes through the private cluster network.

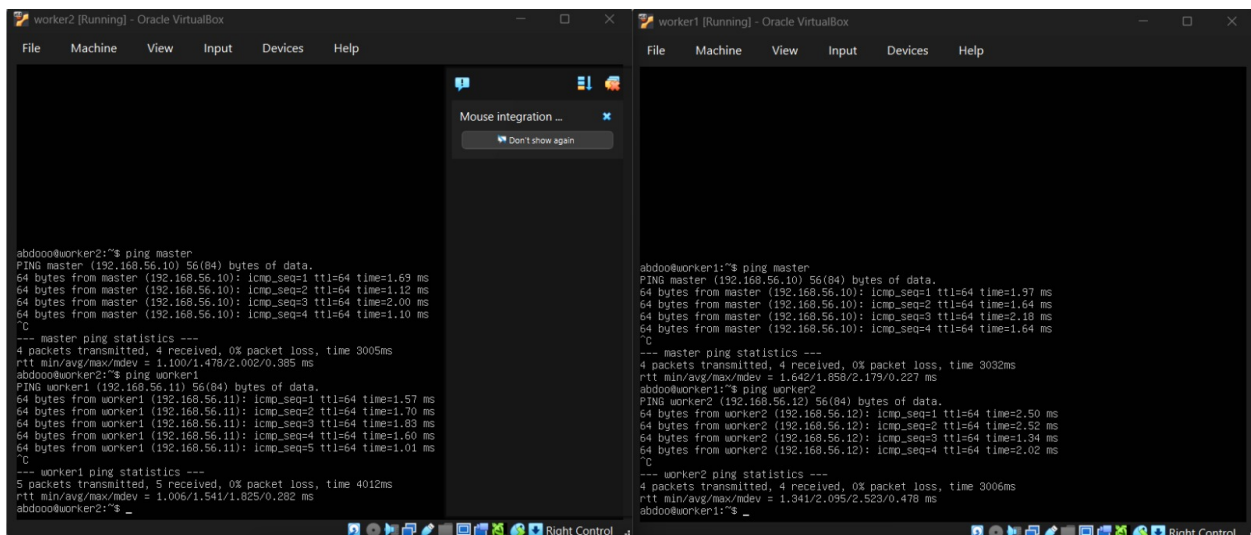


Figure 3: Ping test from master to worker1 & worker2 showing 0% packet loss.

4.Network Troubleshooting Note

During network setup, Netplan configuration errors occurred due to YAML indentation issues and interface activation problems. To troubleshoot this, the Host-only interface was manually enabled using: `sudo ip link set enp0s8 up`

The required IP addresses were then assigned manually using:

```
sudo ip addr add 192.168.56.10/24 dev enp0s8
sudo ip addr add 192.168.56.11/24 dev enp0s8
sudo ip addr add 192.168.56.12/24 dev enp0s8
```

This was done to confirm that the VirtualBox Host-only adapter was working correctly and that the issue was related to configuration formatting rather than the network itself. After assigning the IP addresses, the interfaces were checked using `ip a`, and connectivity was confirmed using `ping`. The successful results with 0% packet loss showed that the three nodes were able to communicate correctly.

5.Step 3: Passwordless SSH Setup

In this step, passwordless SSH was configured between the master node and the two worker nodes. This step is important because the master node needs to connect to the worker nodes automatically during distributed jobs. In MPI-based clusters, the master starts processes on the worker machines through SSH, so passwordless login prevents the job from stopping and waiting for manual password entry.

The SSH setup was performed from the master node. First, an SSH key pair was generated on the master using: `ssh-keygen -t rsa`

During key generation, the default file location was accepted, and no passphrase was added. This created a private key and a public key inside the `.ssh` directory on the master node.

The generated files were:

<code>id_rsa</code>	private	key	stored	on	master
<code>id_rsa.pub</code>	public key copied to workers				

The private key stayed on the master node, while the public key was copied to each worker node. This allows the workers to recognize the master and accept SSH login without asking for a password.

```

abdo@master:~$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/abdo/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/abdo/.ssh/id_rsa
Your public key has been saved in /home/abdo/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:M0I4sMzHL13p0utVkUIk8NgZfsDwqKafLopfgMA32i8 abdo@master
The key's randomart image is:
+---[RSA 3072]-----+
|  . 0+++.0 |
|.o + 0.B . |
|..+0B X o o |
|..+=.* . . . |
|..=.+ + S . |
| o o.. o + |
|. E... . |
|o. o.. . |
|++= . |
+-----[SHA256]-----+
abdo@master:~$ _

```

```

|. E... . |
|o. o.. . |
|++= . |
+-----[SHA256]-----+
abdo@master:~$ ssh-copy-id abdo@worker1
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/abdo/.ssh/id_rsa.pub"
The authenticity of host 'worker1 (192.168.56.11)' can't be established.
ECDSA key fingerprint is SHA256:21ALncqyjMB96pYxsAEpLxy7BBCvSPv79WJKDIgcFvA.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are alr
eady installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to inst
all the new keys
abdo@worker1's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'abdo@worker1'"
and check to make sure that only the key(s) you wanted were added.

abdo@master:~$ ssh-copy-id abdo@worker2
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/abdo/.ssh/id_rsa.pub"
The authenticity of host 'worker2 (192.168.56.12)' can't be established.
ECDSA key fingerprint is SHA256:Ao4wg9MeMLg3GdL1wFXiPnKJGKMBtOK+Kr3BQhLQHpY.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are alr
eady installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to inst
all the new keys
abdo@worker2's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'abdo@worker2'"
and check to make sure that only the key(s) you wanted were added.

abdo@master:~$

```

After generating the SSH key, the public key was copied from the master node to worker1 and worker2 using:

```
ssh-copy-id                                abdoo@worker1  
ssh-copy-id abdoos@worker2
```

The worker password was entered one final time during this step. After the key was copied successfully, the workers added the master public key to their `authorized_keys` file. This means the workers can now trust login requests coming from the master node.

Passwordless SSH login was then tested from the master node.

The connection to worker1 was tested using:

```
ssh                                abdoo@worker1  
hostname  
exit
```

The command successfully logged into worker1 without asking for a password, and the `hostname` output confirmed that the session was inside the correct worker node: `worker1`

The connection to worker2 was tested using the same method. The login also succeeded without asking for a password, and the `hostname` output confirmed that the session was inside worker2.

```
master [Running] - Oracle VirtualBox
Usage of /:                27.2% of 9.75GB
Memory usage:              5%
Swap usage:                0%
Processes:                 117
Users logged in:           1
IPv4 address for enp0s3:   10.0.2.15
IPv6 address for enp0s3:   fd17:625c:f037:2:a00:27ff:fe99:869a
IPv4 address for enp0s8:   192.168.56.11

* Ubuntu 20.04 LTS Focal Fossa has reached its end of standard support
  on 31 May 2025.

  For more details see:
  https://ubuntu.com/20-04

* Introducing Expanded Security Maintenance for Applications.
  Receive updates to over 25,000 software packages with your
  Ubuntu Pro subscription. Free for personal use.

  https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Mon May 25 17:05:09 2026 from 192.168.56.10
abdoo@worker1:~$ _

master [Running] - Oracle VirtualBox
Swap usage:                0%
Processes:                 124
Users logged in:           1
IPv4 address for enp0s3:   10.0.2.15
IPv6 address for enp0s3:   fd17:625c:f037:2:a00:27ff:fed1:a7d
IPv4 address for enp0s8:   192.168.56.12

* Ubuntu 20.04 LTS Focal Fossa has reached its end of standard support
  on 31 May 2025.

  For more details see:
  https://ubuntu.com/20-04

* Introducing Expanded Security Maintenance for Applications.
  Receive updates to over 25,000 software packages with your
  Ubuntu Pro subscription. Free for personal use.

  https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Mon May 25 15:32:54 2026
abdoo@worker2:~$ hostname
worker2
abdoo@worker2:~$ _
```

Figure 4: Passwordless SSH login from master to worker1 and worker2, showing successful hostname verification.

6.Step 4: OpenMPI and Python Environment Setup

After completing the SSH configuration, the required software environment was installed on all three nodes: master, worker1, and worker2. This step was important because the cluster needs the same MPI and Python environment on every node to run distributed jobs successfully.

OpenMPI was installed to allow parallel execution across the cluster nodes. Python and the required scientific libraries were also installed to support the machine learning and bioinformatics scripts.

The following packages were installed on each node:

```
openmpi-bin
libopenmpi-dev
python3
python3-pip
mpi4py
scikit-learn
pandas
numpy
```

The main commands used were:

```
sudo apt update
sudo apt install openmpi-bin libopenmpi-dev -y
sudo apt install python3 python3-pip -y
pip3 install mpi4py scikit-learn pandas numpy
```

After installation, OpenMPI was checked using:

```
mpirun --version
```

Python libraries were also tested to make sure that the required packages were available. This confirmed that the software environment was ready for distributed computing.

7.Step 5: Hostfile Configuration

A hostfile was created on the master node to define the nodes that will participate in MPI jobs. The hostfile tells OpenMPI which machines are available and how many CPU slots can be used on each machine.

The hostfile contained the following configuration:

master	slots=2
worker1	slots=2
worker2	slots=2

This means that each node contributed 2 processing slots. Therefore, the cluster had a total of 6 MPI slots:

2 slots on master + 2 slots on worker1 + 2 slots on worker2 = 6 total slots

The hostfile allowed MPI jobs to be launched from the master node and distributed across the worker nodes.

8. Dataset Description

8.1 Digits Dataset

The digits dataset is a built-in dataset from the scikit-learn library used in the first distributed MPI job (`distributed_mnist.py`). It contains images of handwritten digits represented as numerical feature values, with each sample belonging to one of ten classes (digits 0–9). The dataset was loaded directly in the script using scikit-learn and broadcast from the master process to all MPI processes.

8.2 Gene Expression Dataset (`leukemia_expression.csv`)

The gene expression dataset was created specifically for this project and saved as `leukemia_expression.csv`. According to the report, it contains 200 samples and 20 gene expression features, along with a diagnosis label used for classification. This dataset was used in both the MPI bioinformatics job (`distributed_gene_analysis.py`) and the PySpark job (`distributed_gene_expression_analysis.py`). The file was stored in `/tmp/hpc/` on the cluster nodes and in the Spark master container during the respective jobs.

9. Step 6: Distributed Machine Learning Job

The first distributed job was a machine learning task using the digits dataset from the scikit-learn library. The script was named:

`distributed_mnist.py`

The script used MPI through the `mpi4py` library. The master process loaded the dataset and broadcasted it to all MPI processes. The data was then divided between the 6 MPI processes. Each process trained a Random Forest classifier on its local part of the dataset and calculated a local score.

The purpose of this step was to show that the cluster could run one Python program in parallel across multiple virtual machines.

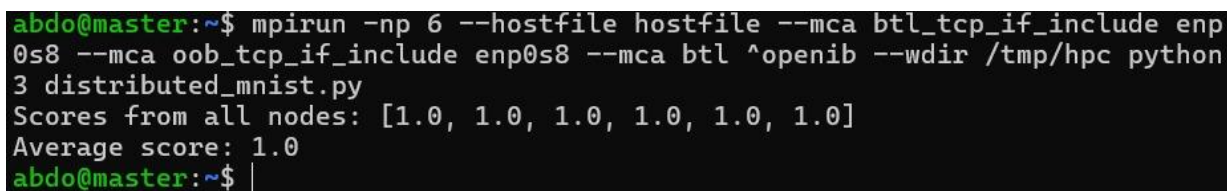
The script was copied to the working directory on all nodes, and the job was executed from the master node using:

```
mpirun -np 6 --hostfile hostfile --mca btl_tcp_if_include enp0s8 --mca oob_tcp_if_include enp0s8 --mca btl ^openib --wdir /tmp/hpc python3 distributed_mnist.py
```

The output was:

```
Scores from all nodes: [1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
Average score: 1.0
```

This result confirmed that all 6 MPI processes ran successfully across the cluster. The score from each process was returned to the master node, and the average score was calculated.



```
abdo@master:~$ mpirun -np 6 --hostfile hostfile --mca btl_tcp_if_include enp0s8 --mca oob_tcp_if_include enp0s8 --mca btl ^openib --wdir /tmp/hpc python3 distributed_mnist.py
Scores from all nodes: [1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
Average score: 1.0
abdo@master:~$
```

Figure 5: Output of the distributed machine learning job showing six node scores and the average score.

10.Step 7: Distributed Bioinformatics Job

The second distributed job was a bioinformatics-based analysis using a gene expression dataset. A CSV file named: leukemia_expression.csv

was created and used as the input dataset. The distributed bioinformatics script was named: distributed_gene_analysis.py

The dataset contained gene expression values and a diagnosis label. The script used MPI to distribute the dataset between the 6 MPI processes. Each process trained a Random Forest classifier on its assigned part of the gene expression data and calculated a classification score.

This step demonstrated how HPC and MPI can be applied to bioinformatics analysis, especially when working with gene expression datasets or large biological data.

The job was executed from the master node using:

```
mpirun -np 6 --hostfile ~/hostfile --mca btl_tcp_if_include enp0s8 --mca oob_tcp_if_include enp0s8 --mca btl ^openib --wdir /tmp/hpc python3 distributed_gene_analysis.py
```

The output was:

```
Bioinformatics scores from all nodes: [1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
Average score: 1.0
```

This confirmed that the bioinformatics job ran successfully on the mini-HPC cluster. All 6 MPI processes completed their tasks and returned their results to the master node.

```
abdo@master:/tmp/hpc$ mpirun -np 6 --hostfile ~/hostfile --mca btl_tcp_if_in  
clude enp0s8 --mca oob_tcp_if_include enp0s8 --mca btl ^openib --wdir /tmp/h  
pc python3 distributed_gene_analysis.py  
Bioinformatics scores from all nodes: [1.0, 1.0, 1.0, 1.0, 1.0, 1.0]  
Average score: 1.0
```

Figure 6: Output of the distributed bioinformatics job showing six scores and the average score.

11.Step 8: Docker Installation and Docker Swarm Setup

After completing the MPI-based Mini-HPC part, the second part of the project focused on building a hybrid HPC and Big Data cluster using Docker Swarm and Apache Spark. Docker was installed on the three main cluster nodes: master, worker1, and worker2.

Docker was installed using the following commands:

```
sudo apt update  
sudo apt install docker.io -y  
sudo systemctl enable --now docker  
sudo docker --version
```

After Docker was installed successfully, Docker Swarm was initialized on the master node using: `sudo docker swarm init --advertise-addr 192.168.56.10`

This command created a Docker Swarm cluster with the master node acting as the Swarm manager. A join token was then generated and used to add worker1 and worker2 to the Swarm cluster.

The Swarm worker nodes joined the cluster using a command similar to: `sudo docker swarm join --token <TOKEN> 192.168.56.10:2377`

The Swarm cluster was verified from the master node using: `sudo docker node ls`

The output showed that all three nodes were active in the Docker Swarm cluster:

master	Ready	Active	Leader
worker1		Ready	Active
worker2	Ready	Active	

This confirmed that the Docker Swarm cluster was successfully created using the three virtual machines.

```

abdo@master:~$ sudo docker node ls
[sudo] password for abdo:
ID                                HOSTNAME    STATUS    AVAILABILITY    MANAGER STATE
ATUS    ENGINE VERSION
ku7ct91esh8dzf1c4uoehimv3 *    master     Ready     Active           Leader
26.1.3
owjuv471aord4whpxqn66wx06      worker1    Ready     Active
26.1.3
o5e0v35mw9w4wzjzoanwg24sj      worker2    Ready     Active
26.1.3
abdo@master:~$

```

Figure 7: Docker Swarm node list showing master, worker1, and worker2 in Ready state.

12.Step 9: Spark Cluster Deployment Using Docker Swarm

After the Docker Swarm cluster was created, Apache Spark was deployed as a distributed service using a Docker stack file named: spark-stack.yml

The Spark cluster included one Spark master service and two Spark worker replicas. The official Spark Docker image was used: spark:3.5.0

The Spark stack file defined the Spark master and Spark worker services. The Spark master exposed ports 7077 and 8080, while the Spark workers connected to the master using: spark://spark-master:7077

The stack was deployed using: `sudo docker stack deploy -c spark-stack.yml hpc`

The Spark services were checked using: `sudo docker service ls`

The final output showed:

```

hpc_spark-master  1/1
hpc_spark-worker  2/2

```

This confirmed that the Spark master and two Spark worker replicas were running successfully inside the Docker Swarm environment.

The stack status was also checked using: `sudo docker stack ps hpc`

This command showed the running Spark containers and confirmed that the Spark cluster was active.

```
abdo@master:~$ sudo docker service ls
ID                NAME                MODE                REPLICAS    IMAGE                PORT
rxihtukcijhz      hpc_spark-master    replicated          1/1          spark:3.5.0         *:70
77->7077/tcp, *:8080->8080/tcp
cu8uj045424o      hpc_spark-worker    replicated          2/2          spark:3.5.0
abdo@master:~$ sudo docker stack ps hpc
ID                NAME                IMAGE                NODE          DESIRED STATE
CURRENT STATE    ERROR              PORTS
0s1laaffx17yg     hpc_spark-master.1  spark:3.5.0         master       Running
Running 5 minutes ago
miwex7mdje7j      hpc_spark-worker.1  spark:3.5.0         master       Running
Running 5 minutes ago
lfhadwysadoy      hpc_spark-worker.2  spark:3.5.0         master       Running
Running 5 minutes ago
abdo@master:~$ |
```

Figure 8: Docker service list showing Spark master 1/1 and Spark worker 2/2 and Spark services running.

Spark Deployment Troubleshooting Note

During Spark deployment, the original Bitnami Spark image caused image availability issues. To solve this, the official Spark image was used instead: spark:3.5.0

Another issue occurred when some Spark worker containers failed with a user-related error. This was solved by running the Spark containers as root inside the Docker stack file using: user: "0" .The worker memory was also limited to 512 MB to match the available VM resources: --memory 512m ,After these changes, the Spark master and two Spark worker replicas ran successfully.

13.Step 10: Distributed Spark Bioinformatics Machine Learning Job

After the Spark cluster was deployed successfully, a PySpark bioinformatics machine learning job was executed. The script was named: distributed_gene_expression_analysis.py

A gene expression dataset file named: leukemia_expression.csv, was used as the input dataset. The dataset contained 200 samples and 20 gene expression features, with a diagnosis label used for classification.

The Spark job was submitted from inside the Spark master container using:

```

sudo docker exec -it $MASTER_ID /opt/spark/bin/spark-submit \
--master spark://spark-master:7077 \
--executor-memory 512m \
--driver-memory 512m \
--total-executor-cores 2 \
/tmp/distributed_gene_expression_analysis.py

```

The Spark output showed that the job connected successfully to the Spark cluster. Two Spark executors were added, each with 1 core and 512 MB memory. The tasks were then distributed and executed across the Spark executors.

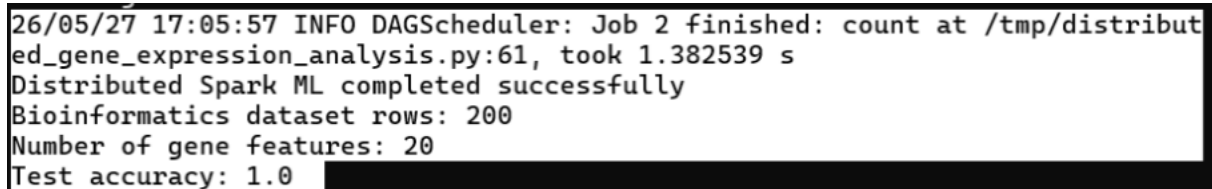
The final output was:

```

Distributed Spark ML completed successfully
Bioinformatics dataset rows: 200
Number of gene features: 20
Test accuracy: 1.0

```

This confirmed that the distributed Spark machine learning job completed successfully. The result demonstrated that Docker Swarm and Spark could be used to run a distributed bioinformatics machine learning workflow.



```

26/05/27 17:05:57 INFO DAGScheduler: Job 2 finished: count at /tmp/distributed_gene_expression_analysis.py:61, took 1.382539 s
Distributed Spark ML completed successfully
Bioinformatics dataset rows: 200
Number of gene features: 20
Test accuracy: 1.0

```

Figure 9: PySpark job output showing successful execution and test accuracy of 1.0.

14. Results Analysis

14.1 Summary of Reported Results

Job	Script	Processes	Result
Distributed digit classification	distributed_mnist.py	6 MPI	All scores: 1.0 — Average: 1.0
Distributed gene expression (MPI)	distributed_gene_analysis.py	6 MPI	All scores: 1.0 — Average: 1.0
Distributed gene expression (Spark)	distributed_gene_expression_analysis.py	2 Spark executors	Test accuracy: 1.0

14.2 Discussion of Perfect Scores

All jobs returned a score of exactly 1.0 across every process and executor. While this confirms the jobs ran and completed successfully, the perfect scores require careful interpretation:

Very small per-process test sets

The MPI jobs split the dataset across 6 processes. For the gene expression dataset (200 samples), each process received approximately 33 samples, of which only 20% — around 6 to 7 samples — were used for testing. A test set this small is not statistically reliable and can easily yield 100% accuracy by chance, especially on a dataset that was synthetically generated for this project.

Synthetic dataset

The leukemia_expression.csv dataset was created for the project rather than taken from a real scientific source. Synthetically generated datasets are often linearly separable by design, making them straightforward for a Random Forest classifier to classify perfectly even on very small test partitions.

No global holdout set

Each MPI process evaluated its model only on its own local data slice. There was no shared global test set that all processes were evaluated against. This means the reported scores reflect local performance on small subsets, not the overall generalisation ability of the distributed models.

What this means

The results confirm that the cluster infrastructure worked correctly and that jobs were successfully distributed. However, the accuracy figures of 1.0 should not be interpreted as a meaningful measure of model quality. A proper evaluation would require a fixed global test set, cross-validation, and a real-world dataset.

15. Limitations

15.1 VM-Based Cluster

All three nodes ran as virtual machines on the same physical host. This means they shared the same physical CPU, RAM, and disk. As a result, the cluster cannot demonstrate real parallel speedup — any gains from distributing work across nodes are offset by the overhead of VM scheduling and virtual network communication. The project is appropriate for learning HPC concepts but does not reflect the performance of a real physical cluster.

15.2 No Execution Time Measurement

The report does not record or compare execution times for any of the jobs. Without timing data, it is not possible to assess whether distributing the work across 6 MPI processes or 2 Spark executors resulted in any speedup compared to running the same job on a single process.

15.3 Missing Screenshots

The report references nine figures (Figures 1–9) that are described in captions but do not appear in the document. These screenshots were intended to serve as evidence for the networking setup, SSH configuration, MPI job output, Docker Swarm status, and Spark job results. Their absence means the report's claims cannot be visually verified from the document alone.

15.4 Username Inconsistency in SSH Commands

The SSH commands used during passwordless setup show two different usernames: `ssh-copy-id abdoo@worker1` and `ssh-copy-id abdoos@worker2`. This inconsistency is not acknowledged in the report and may indicate a typographical error or that the actual usernames on the two worker nodes differed during setup.

16. MPI Troubleshooting Note

During the first MPI run, a communication error appeared because OpenMPI tried to use more than one network interface. The error was related to MPI communication between the master and worker nodes.

To solve this problem, MPI was forced to use the Host-only network interface only. The Host-only interface used for cluster communication was: `enp0s8`

The following MPI options were added to the `mpirun` command:

```
--mca                                btl_tcp_if_include          enp0s8
--mca                                oob_tcp_if_include          enp0s8
--mca btl ^openib
```

After adding these options, the MPI jobs ran successfully. This confirmed that the issue was not in the Python scripts, but in the MPI network interface selection.

17. Conclusion

This project successfully built and operated a Mini-HPC cluster using three VirtualBox virtual machines running Ubuntu Server 20.04. All infrastructure components were configured and verified: the private Host-only network allowed the nodes to communicate, passwordless SSH allowed the master to launch jobs on the workers, and both the MPI and Docker Swarm/Spark frameworks ran distributed jobs across the cluster.

Two MPI-based machine learning jobs and one PySpark job completed successfully, with results gathered at the master node in each case. The troubleshooting steps documented — including Netplan YAML

errors, MPI interface binding, and Spark image compatibility issues — demonstrate practical experience with distributed system configuration.

The main limitation of the project is that all three nodes shared a single physical machine, which prevents any real measurement of parallel performance. The perfect accuracy scores reported across all jobs are a consequence of very small per-process test sets and a synthetic dataset, rather than an indication of model quality. Future work could address this by using physical or cloud-based nodes, measuring execution times, and evaluating models on real-world datasets with proper validation methodology.

18. Final Project Files

The final project folder was prepared with the required files:

```
project_hpc_hybrid_cluster/  
  hostfile  
  distributed_mnist.py  
  distributed_gene_analysis.py  
  bioinfo_data/  
  leukemia_expression.csv  
  screenshots/  
  setup_log.txt  
  Final_Report.md
```

The screenshots folder contains the required screenshots for the project, including networking, SSH, and job outputs. The setup_log.txt file contains the main commands and outputs used during the project.

19. References

1. Open MPI Project. Open MPI: Open Source High Performance Computing. <https://www.open-mpi.org/>
2. mpi4py Development Team. mpi4py — MPI for Python. <https://mpi4py.readthedocs.io/>
3. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
4. Apache Software Foundation. Apache Spark Documentation v3.5.0. <https://spark.apache.org/docs/3.5.0/>
5. Docker Inc. Docker Swarm Documentation. <https://docs.docker.com/engine/swarm/>

6. Canonical Ltd. Ubuntu Server 20.04 LTS Documentation. <https://ubuntu.com/server/docs>
7. Oracle Corporation. Oracle VM VirtualBox User Manual. <https://www.virtualbox.org/manual/>